Hidden errors

Finding errors in code

D3 Silent errors

D3-L1 Join row explosion and nulls in filters

Check row counts across every join. Know how your engine treats nulls in filters and keys, and write the null case explicitly.

Check for join row explosion	Row counts before and after. pandas: validate='many_to_one'. Spark: count duplicate keys in the lookup first.
col != 'x' with nulls: pandas vs Spark	pandas keeps missing values. Spark and SQL drop nulls, because the comparison is null.
Null join keys: pandas vs Spark	pandas merge matches null to null. Spark and SQL never match null keys.
NOT IN with a null in the list	Returns no rows. Use NOT EXISTS or a left_anti join.

D3-L2 Type coercion and timezone drift

Read identifiers as strings, union by name, and keep every timestamp aware and stored in UTC.

pd.read_csv and leading zeros	Default parsing makes 0123 into 123. Pass dtype='string' for identifiers.
Spark union vs unionByName	union matches columns by position, silently. unionByName matches by name.
datetime.utcnow()	Deprecated from Python 3.12 and naive. Use datetime.now(timezone.utc).
Spark TIMESTAMP and the session time zone	An instant, shown and converted in spark.sql.session.timeZone. to_date can move a row to another day.

D3-L3 Mutable defaults and the truthiness of zero

Both fixes are one line. Both bugs pass every test that uses only non-zero values and a single call.

```python
def add_check(result: str, results: list[str] | None = None) -> list[str]:
    results = [] if results is None else results
    results.append(result)
    return results
```

```python
if current_pct and prior_pct:
```

```python
if current_pct is not None and prior_pct is not None:
```

When does Python evaluate a default argument?	Once, when it defines the function. A list or dict default is shared by every call.
Safe pattern for an optional list argument	Default to None. Inside: results = [] if results is None else results.
What is wrong with `if current_pct and prior_pct:`?	0.0 is falsy, so a real share of zero is treated as missing. Use `is not None`.

Type hint for an optional string that defaults to None	str | None = None. Never str = None.

D4 Data errors

D4-L1 Row counts across layers

Write the expected relationship between layers as rules, compute counts in one pass on identical filters, and halt when a CRITICAL rule fails.

Row-count rules: silver vs bronze	Silver rows = distinct respondent keys in bronze for the tracker and wave.
Rows after a MERGE	Rows before + numTargetRowsInserted (minus any deletes). Updates add nothing.
Counting efficiently	One agg pass: F.count('*') and F.count_distinct(key). Not repeated count() calls.
The get_unprocessed_waves bug	Bronze filtered on tracker, silver not. Filter both sides on the same keys.

D4-L2 Distribution drift between waves

Compare shares over the union of categories, flag new and missing values, summarise with PSI, and guard every total.

```
PSI = sum over categories of (actual - expected) x ln(actual / expected)
```

Three drift checks	Category shares vs prior wave, new and missing categories, and a PSI summary.
PSI formula	Sum of (actual - expected) x ln(actual / expected) over shares, with zero shares floored.
PSI thresholds 0.1 and 0.25	A rule of thumb, not a standard. Calibrate on your own wave history.
Percentage points vs relative change	30% to 45% is +15 points and +50% relative. Say which a tolerance uses.

D4-L3 Weight totals and survey checks

Check nulls, range, sum and efficiency per wave and subgroup. Mind Spark's nulls: sample stddev, empty aggregates and sums over no matches.

```
deff  = n x sum(w^2) / (sum w)^2 = 1 + CV^2
n_eff = (sum w)^2 / sum(w^2) = n / deff
```

Kish design effect and effective sample	deff = n x sum(w^2) / (sum w)^2 = 1 + CV^2. n_eff = (sum w)^2 / sum(w^2).
F.stddev in Spark	Sample standard deviation, null for one row. Use F.stddev_pop for 1 + CV^2.
sum(when(cond, 1)) with no matches	Null, not 0. Wrap in coalesce(..., lit(0)) or add otherwise(0).
Severity for a null weight	CRITICAL: halt before gold.